

Proyecto final

IDL: Reconocimiento de extraccion de alarmas

Autor: Maximiliano Torres

Profesor: Gabriel Hermosilla

Índice general

Introducción	1
Objetivo general	1
1 Antecedentes	2
1.1 Problemática	2
1.2 Solución	2
1.2.1 Lógica del programa	2
2 Marco teórico	4
2.1 Roboflow	4
2.2 YOLO v8	4
3 Metodología y resultados	6
3.1 Proceso de implementación y evaluación de la estrategia de detección y conteo	6
3.2 Explicación detallada del algoritmo	8
3.3 Resultados	8
Discusión y conclusiones	14
Bibliografía	15

Introducción

Un área de la inteligencia artificial que ha avanzado muchísimo estos últimos años es el Deep Learning, el cual utiliza redes neuronales profundas para resolver distintos tipos de problemas, desde regresión lineal hasta reconocimiento de imágenes. Uno de los sectores beneficiados con estos algoritmos es la industria del retail, donde la automatización y el análisis de datos es fundamental para optimizar los procesos y los recursos. Además, otro aspecto importante es la seguridad, en donde la implementación de estas tecnologías ayudaría a evitar robos y pérdidas monetarias en la empresa.

En este informe lo que se busca es planear e implementar una estrategia de detección y conteo de prendas que son desalarmadas dentro de una tienda de retail. Dentro de un día, cientos de personas asisten para realizar compras, incluyendo un gran porcentaje en prendas de ropa, por ende, se necesita un método para contabilizar cuantas prendas de ropa se van de la tienda de retail.

En las siguientes secciones, se detallará el proceso de desarrollo del algoritmo, desde el preprocesamiento de las imágenes y la selección del modelo, hasta el entrenamiento y la evaluación del algoritmo.

Objetivo general

- Desarrollar un algoritmo que logre detectar cada vez que se desarma una prenda de vestir.

1 Antecedentes

En esta sección se explicará la problemática del proyecto y se presentará la solución propuesta.

1.1 Problemática

La problemática que se dispone a resolver en este proyecto es detectar y contar cada vez que se desalarma una prenda de ropa en las mesas de autoatención de una tienda de retail, para así verificar que las alarmas están siendo correctamente removidas, que los clientes efectivamente están comprando los productos y no sean robados.

1.2 Solución

La solución que se va a implementar a continuación para la detección y conteo de prendas desalarmadas es aplicar técnicas de Deep learning mediante el uso de un modelo pre-entrenado, que utiliza imágenes de los videos de las cámaras de seguridad ubicadas en las mesas de autoatención, para detectar el momento en que se desalarma una prenda de ropa que es justamente cuando la alarma se ubica sobre el imán.

1.2.1 Lógica del programa

La lógica presente en el programa corresponde a un sistema en el cual se implementan dos modelos de inteligencia artificial distintos: uno para identificar y hacer zoom en el área donde se desea observar el proceso de desalarmado y otro para hacer seguimiento a la posición de la alarma, la visibilidad del imán y el basurero. Con esto último, se realiza posteriormente un script en Python que, basado en las coordenadas de estos tres objetos, es capaz de contabilizar de manera efectiva cuántas prendas se desalarman en un solo video.



Figura 1.1: Funcionamiento visto de forma grafica

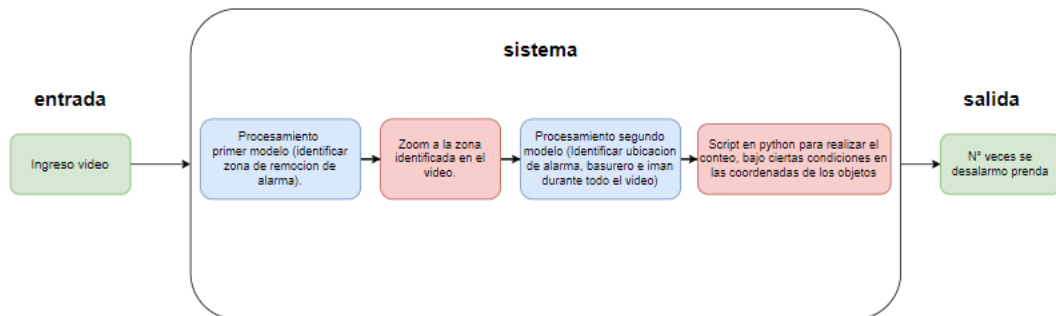


Figura 1.2: Modelado de sistema

2 Marco teórico

2.1 Roboflow

Roboflow es una plataforma web que sirve para facilitar y optimizar el desarrollo de modelos de visión por computadora al proporcionar herramientas de machine learning. Las ventajas que entrega esta plataforma son:

- Eficiencia: Al proporcionar una plataforma integral que abarca desde la carga de imágenes hasta el despliegue del modelo se reduce significativamente el tiempo de creación de modelos de visión por computadora.
- Compatibilidad y flexibilidad de implementación: Permite implementar los modelos en una variedad de entornos, incluyendo Jetson Nano, Raspberry Pi, Azure, etc.
- Integración y personalización sencillas: Ofrece API abiertas, SDK y una amplia documentación que facilita su integración en los procesos de visión por computadora.

Una de las numerosas herramientas que entrega Roboflow es *Annotate*, que permite etiquetar imágenes mediante bounding boxes, polígonos o segmentación y además crear las clases.

2.2 YOLO v8

You Only Look Once es un algoritmo de código abierto, que sirve para detectar objetos en tiempo real utilizando una única red neuronal convolucional. Fue introducido en el año 2015 en el artículo titulado *You Only Look Once: Unified, Real-time Object Detection* [1]. En la Figura 2.1 se puede ver a grandes rasgos como funciona este algoritmo.

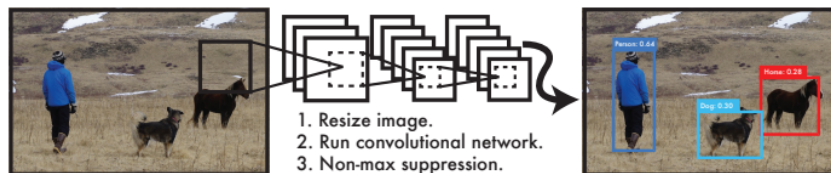


Figura 2.1: Funcionamiento de YOLO

Existen muchas versiones de YOLO, entre ellas se encuentra YOLO V8, esta fue entrenada con

la base de datos COCO de Microsoft y permite realizar tareas de detección de objetos, clasificación y segmentación. Las ventajas que ofrece esta versión son su velocidad, su precisión mejorada en detección y una mejor generalización [2]. YOLO v8 también tiene sus variantes, las que se pueden apreciar en la Figura 2.2. De estas, la que se utilizará es la versión m.

Model	Layers	Size (pixels)	Params (M)	FLOPs (B)
YOLOv8s	225	640	11.2	28.6
YOLOv8m	295	640	25.9	78.9
YOLOv8l	365	640	43.7	165.2

Figura 2.2: Variantes de YOLOv8

3 Metodología y resultados

3.1 Proceso de implementación y evaluación de la estrategia de detección y conteo

Antes de explicar el proceso de implementación y evaluación de la estrategia de detección y conteo, se muestra en la siguiente Figura 3.1 el diagrama de flujo que involucra el paso a paso que se siguió:

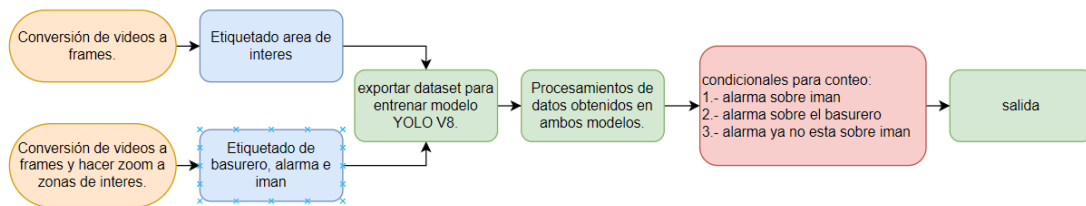


Figura 3.1: Enter Caption

Al inicio de este proyecto, el primer paso a realizar fue anotar en una tabla en Excel (archivo "prendas.csv") la cantidad de prendas desalarmadas en cada uno de los videos almacenados en el link proporcionado por la empresa Soluciones IA SPA. Después, se diseña una estrategia de detección y conteo cada vez que se remueve una alarma de las prendas. La estrategia a implementar empieza en primer lugar al convertir los videos (archivos .mp4) a frames a través de la librería "video2images".

Posteriormente, se procedió a recortar las imágenes obtenidas, centrando únicamente en el área de interés, donde se encuentran el imán y el basurero de alarmas. Luego, por medio de la plataforma web Roboflow, se dibujan los cuadros delimitadores para tres clases definidas: la alarma de ropa (rojo), el imán (amarillo) y el basurero en que se desechan las alarmas (morado). Se priorizan las imágenes en que la alarma esté sobre el imán, y cuando se vaya a botar la alarma al basurero. En la Figura 3.2 se puede ver un ejemplo de cómo se realizaron las etiquetas.

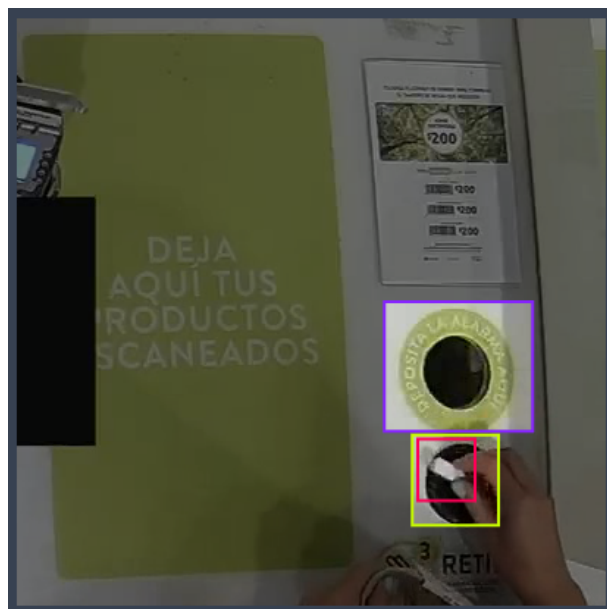


Figura 3.2: Imagen del proceso de desalarmado

Después de la aplicación de los marcos en las imágenes se carga el dataset, el cual finalmente quedó de 5000 imágenes, que después se usa para entrenar el modelo preentrenado a YOLOv8. Luego, se crea un archivo en Google Colab, en este archivo se generarán modelos que sean capaces detectar y contar cada vez que se desarma una prenda, los parámetros del primer modelo usado son 'epochs'(épocas)=10, 'imgsz'(tamaño de imagen)=640 y 'batch'(lote)=20, en tanto que el segundo modelo, se mantuvo 'imgsz', 'epochs'=30 y 'batch'=40.

Paralelamente, se realizó lo propio con el dataset de la zona de interés, dando como resultado un total de 3200 imágenes.

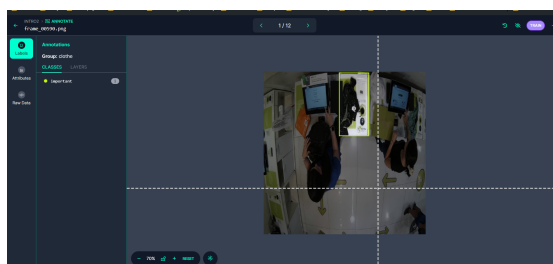


Figura 3.3: dataset

Los parámetros del modelo YOLO utilizado para entrenar este último, corresponde a 'imgsz'=640, 'epochs'=100 y 'batch'=40.

3.2 Explicación detallada del algoritmo

Para el algoritmo de detección y conteo, se utilizaron las salidas del video ya procesado con las respectivas etiquetas y coordenadas de los objetos. Luego, se implementa una lógica simple, en la cual, mediante condicionales, solo se capturan los frames en donde una alarma se halla sobre el imán. Este dato se guarda en un buffer que almacena la cantidad de frames en los que la alarma se encuentra sobre el imán. Luego, este buffer se corta una vez que la alarma es arrojada dentro del basurero o si simplemente deja de estar sobre el imán. Finalmente, cada buffer es almacenado en una lista y, al final del código, se contabiliza qué tan larga es esta lista. De este modo, se evitan los problemas en caso de que una persona deje una alarma sobre un imán, ya que esto simplemente contaría como un solo buffer. Lo mismo ocurre cuando se tapa el basurero o de un momento a otro la alarma desaparece del imán.

3.3 Resultados

A continuación se presentan los resultados como la matriz de confusión y otras métricas de rendimiento (precisión, recall, F1-score) de las predicciones de las ubicaciones de los objetos, no de la salida del modelo. La siguiente Figura 3.4 muestra la matriz de confusión:

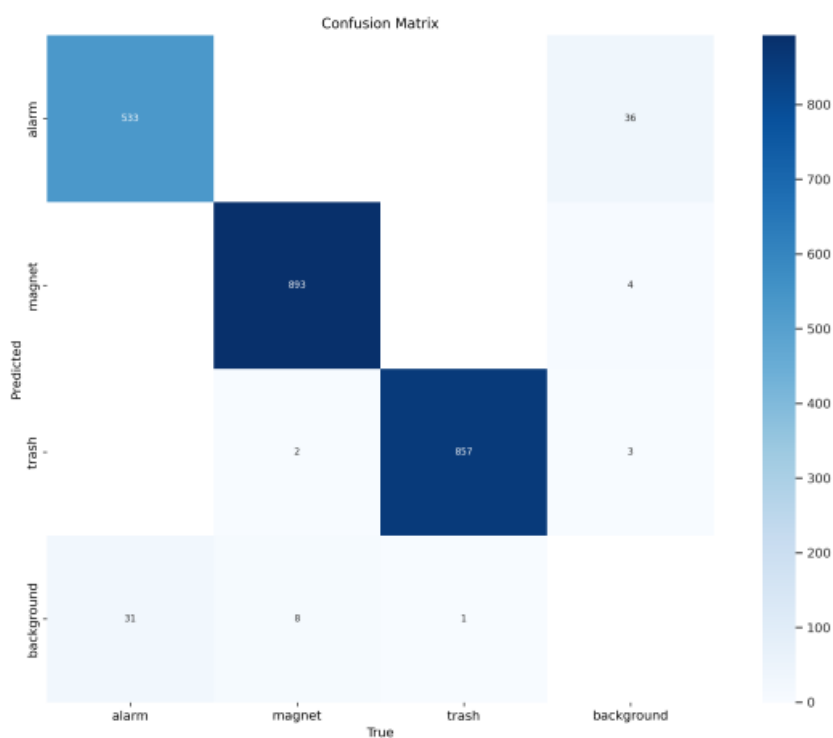


Figura 3.4: Matriz de confusión

De la Figura 3.4 se puede decir que de cada 100 muestras 53 muestras de 'alarms' son verda-

deras, en tanto que de cada 100 muestras de 'magnet' 89 son verdaderas y de 100 muestras de 'trash', 85 son verdaderas. Ahora, en la siguiente Figura 3.5, se presenta la curva de recall:

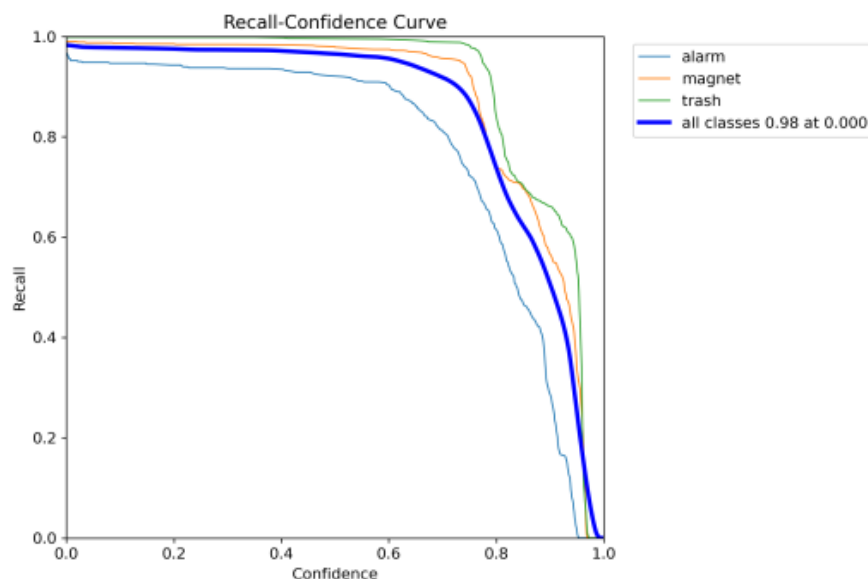


Figura 3.5: Gráfica de Recall confidence

La curva de Recall-Confidence muestra la relación entre los verdaderos positivos y falsos negativos. Mientras más elevada sea la pendiente de la curva mayor será la credibilidad de los resultados de cada clase, en este caso, la clase 'trash' supera levemente en credibilidad a la clase 'magnet' aunque ambas poseen una muy buena credibilidad, en tanto la clase 'alarm' es la que menos veracidad posee, enseguida, se muestra la gráfica de F1-Score en la siguiente Figura 3.6:

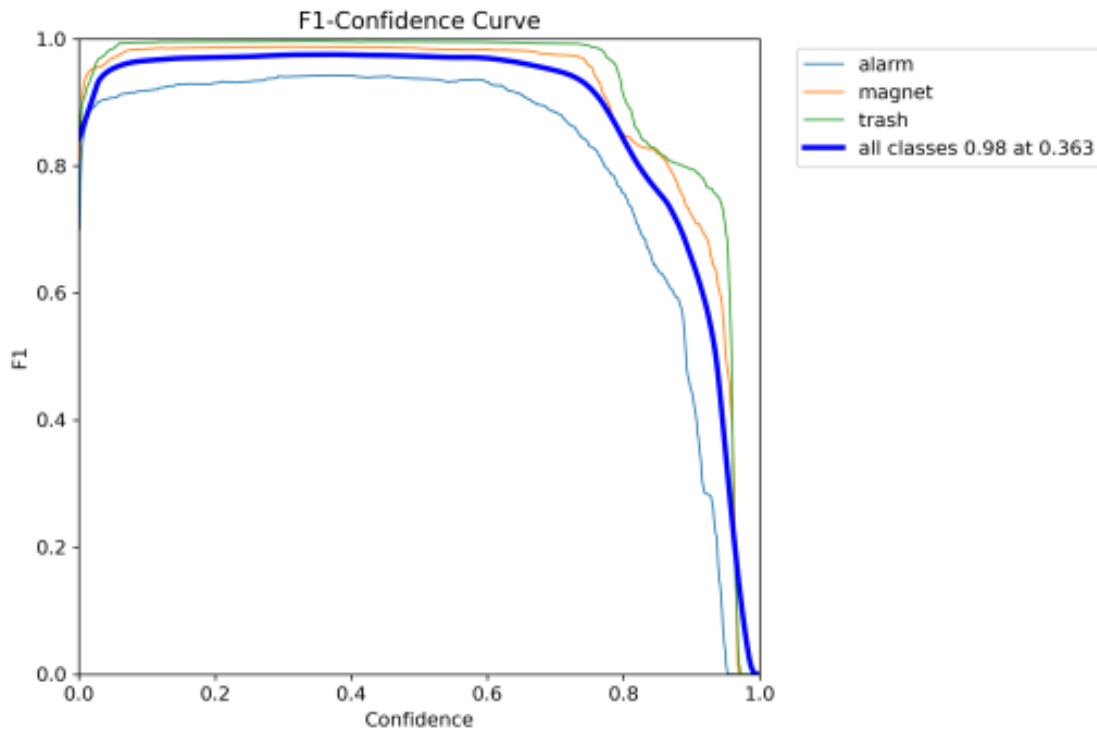


Figura 3.6: Gráfica de F1-Score

La curva de F1-Score significa, en este caso, que mientras mayor sea la pendiente de las curvas mayor será la probabilidad de que se detecten las clases dentro de un frame, mientras que menor sea la pendiente menos probabilidades serán que se detecten las clases, en esta gráfica, se aprecia que las clases 'trash' y 'magnet' poseen altas probabilidades de ser detectadas en los frames a diferencia de la clase 'alarm' cuya probabilidad es menor a la de las otras dos. A continuación en la Figura 3.7 se muestra la gráfica de Precision-Recall:

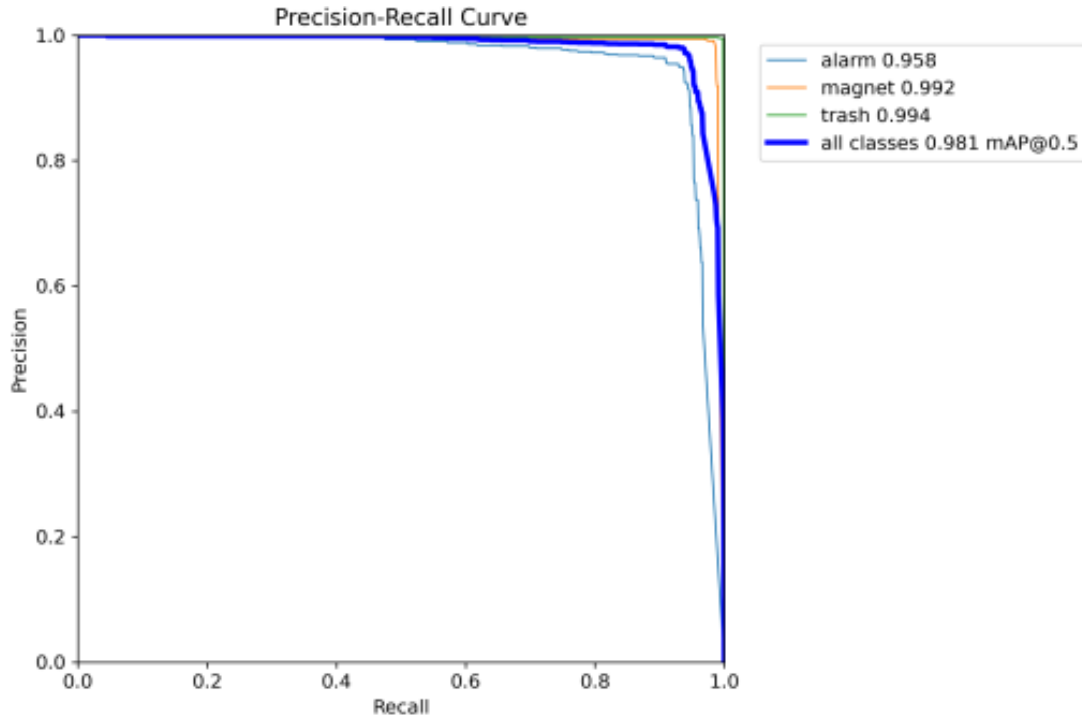


Figura 3.7: Gráfica de Precision-Recall

La curva de Precision Recall relaciona los verdaderos positivos con los falsos positivos. Mientras mayor sea la pendiente de la curva mayor credibilidad tendrá la clase, en tanto si es menor la pendiente menos credibilidad tendrá. En la Figura 3.7 las clases 'magnet' y 'trash' poseen una muy buena credibilidad mientras que la clase 'alarm' también la posee aunque sea la menor entre las tres clases.

Finalmente, a continuación se presentan los resultados y métricas obtenidos del sistema una vez implementado en todos los videos:

A continuación, la matriz de confusión de conteos exitosos de cada video:

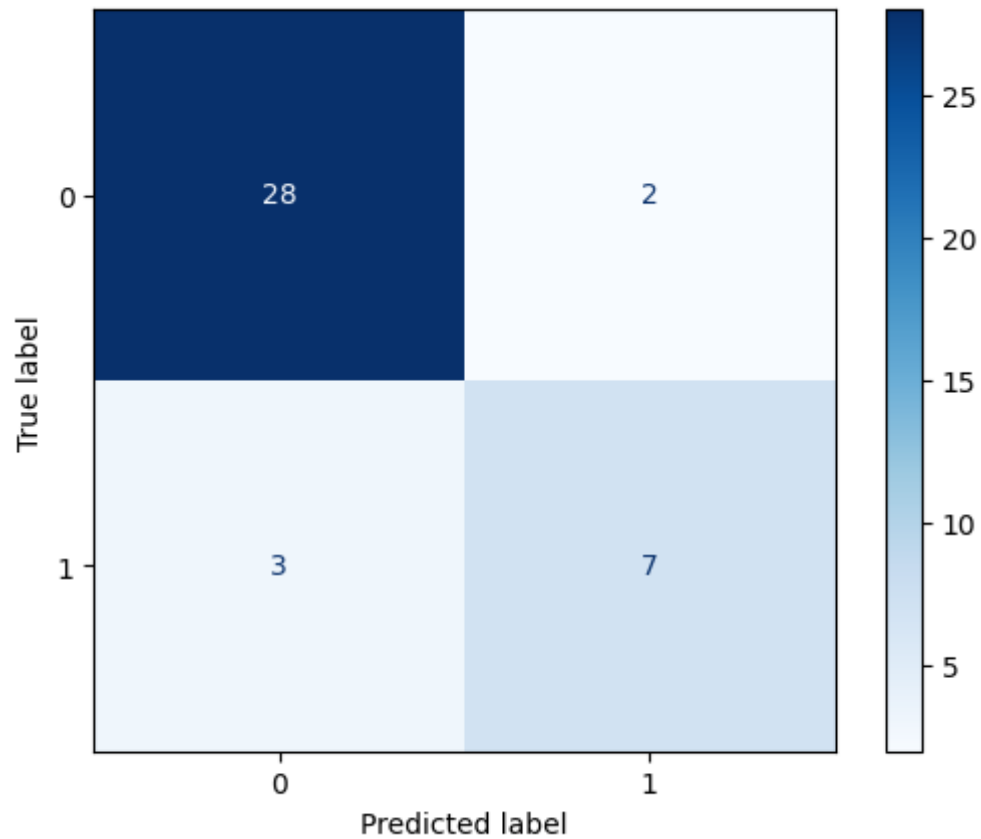


Figura 3.8: Matriz de confucion

Y a continuación, una tabla con las métricas:

Tabla 3.1: Metricas

Metrica	Resultado
Accuracy	87.5 %
Precision	0.93 [-]
Recall	0.90 [-]
F1score	0.918 [-]

Y adicionalmente se agrego una abla hecha en excel que muerta los resultado obtenidos en cada video:

3 Trabajo de investigación

Video	Conteo Real	Predicho	True Positive	True Negative	False Negative	False Positive
Test 1	1	1	1	0	0	0
Test 2	2	2	2	0	0	0
Test 3	2	4	2	0	0	2
Test 4	0	0	0	1	0	0
Test 5	2	2	2	0	0	0
Test 6	2	2	2	0	0	0
Test 7	0	0	0	1	0	0
Test 8	0	0	0	1	0	0
Test 9	0	0	0	1	0	0
Test 10	3	3	3	0	0	0
Test 11	0	0	0	1	0	0
Test 12	5	3	3	0	2	0
Test 13	0	0	0	1	0	0
Test 14	0	0	0	1	0	0
Test 15	2	2	2	0	0	0
Test 16	5	4	4	0	1	0
Test 17	2	2	2	0	0	0
Test 18	2	2	2	0	0	0
Test 19	2	2	2	0	0	0
Test 20	1	1	1	0	0	0
Total:	31	30	28	7	3	2

Figura 3.9: Tabla excel Resultados

Discusión y conclusiones

Respecto a los valores y gráficas de las métricas de rendimiento y matriz de confusión se puede concluir, que entre las tres clases ('alarm', 'magnet', 'trash') la clase 'alarm' es la que menos veracidad tiene siendo más visible la diferencia en la matriz de confusión. Para mejorar la robustez de este modelo, una solución puede ser no haber recortado las imágenes (frames) de los videos y realizar el premodelamiento de imágenes con la imagen completa, y la otra solución es, usar YOLO para encontrar características de otros modelos que son las coordenadas de los 'bounding boxes' de las clases. Esto implica que el modelo aprenda de manera autónoma a establecer cuando se ubica la alarma ('alarm') sobre el imán ('magnet'), lo cual vendría siendo un sistema de conteo.

En cuanto a los entrenamientos de los modelos, existen probabilidades de que los resultados obtenidos en las métricas de rendimiento y matriz de confusión varíen si en el entrenamiento de los modelos se implementan más variaciones de parámetros ('batch' y 'epochs' principalmente).

En última instancia, la implementación exitosa de este sistema de detección y conteo automatizado no solo optimizaría la operación de las tiendas al reducir las pérdidas por robo, sino que también ejemplificaría el poder transformador del área del Deep Learning en la industria del retail. Sin embargo, alcanzar este objetivo requiere un enfoque continuo en la optimización del modelo, respaldado por un entendimiento profundo de las limitaciones actuales y una disposición para iterar y mejorar sobre la base de datos empíricos y avances tecnológicos.

Bibliografía

- [1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” 2015.
- [2] Ultralytics, “Explore Ultralytics YOLOv8.” [En línea]. Disponible en: <https://yolov8.com/>